

train_test_split

$X = [\blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare]$ (100%)

$y = [\text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯}]$ (100%)



▼ `train\_test\_split(X, y, test\_size=0.2)` 실행!



$X_{\text{train}} = [\blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare \blacksquare]$ (80%)

$y_{\text{train}} = [\text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯} \text{🎯}]$ (80%)

훈련용
데이터

$X_{\text{test}} : [\blacksquare \blacksquare]$ (20%)

$y_{\text{test}} : [\text{🎯} \text{🎯}]$ (20%)

검증용
데이터

StandardScaler

역할: 데이터의 단위(스케일)를 맞춰서 공평하게 만들어주는 도구

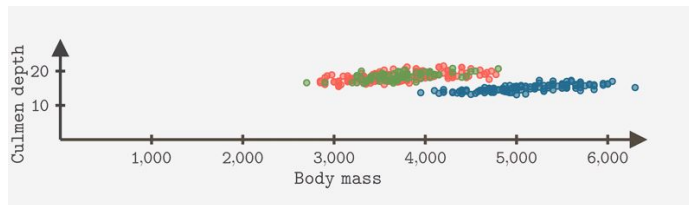
💡 왜 필요할까?

만약 사람의 건강 데이터를 가지고 질병을 예측하는데, 어떤 데이터는 '키(cm)'이고 어떤 데이터는 '몸무게(kg)'라고 가정.

- 키는 대략 150 ~ 190 사이의 값
- 몸무게는 40 ~ 100 사이의 값.

만약 키 170, 몸무게 70인 데이터에서
컴퓨터는 숫자의 크기만 보고 "와, 키는 170이니까 몸무게보다 2.5배 더 중요한 데이터구나 !" 하고 판단.

이런 오해를 막기 위해 모든 데이터의 평균을 0, 표준편차를 1로 만들어서 치우침 없이 공평한 저울 위에 올려놓는 작업(표준화)을 수행



StandardScaler 더 알아보기

<https://codefinity.com/courses/v2/a65bbc96-309e-4df9-a790-a1eb8c815a1c/1fce4aa9-710f-4bc9-ad66-16b4b2d30929/79d587a4-bba9-45c8-878f-f2948f0b0c7e>

DecisionTreeClassifier

역할: 스무고개처럼 질문을 던지며 정답을 맞춰가는 분류 모델

💡 왜 필요할까?

데이터를 다 고르고 나면 컴퓨터가 학습 필요

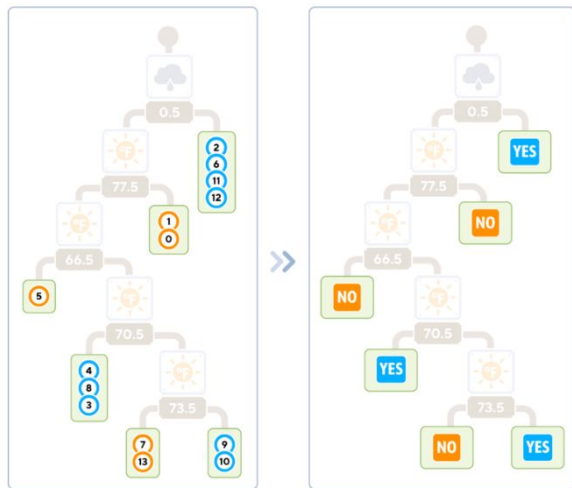
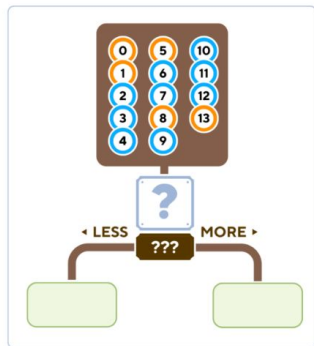
DecisionTreeClassifier는 데이터를 학습해서 스무고개 지도(Tree)를 그리는 인공지능 모델.

예를 들어, "이 동물이 타이거(호랑이)일까, 고양이일까?"를 분류할 때 컴퓨터는 아래와 같이 판단을 수행.

1. "몸무게가 50kg 이상인가요?" → Yes / No
2. (Yes인 경우) "줄무늬가 있나요?" → Yes (호랑이!) / No (사자!)

Training Set

0	...	NO
1	...	NO
2	...	YES
3	...	YES
4	...	YES
5	...	NO
6	...	YES
7	...	YES
8	...	NO
9	...	YES
10	...	YES
11	...	YES
12	...	YES
13	...	NO



DecisionTreeClassifier 더 알아보기

<https://towardsdatascience.com/decision-tree-classifier-explained-a-visual-guide-with-code-examples-for-beginners-7c863f06a71e/>

Pipeline

역할 : 머신러닝의 '자동화 컨베이어 벨트'.

앞에서 배운 데이터 전처리(**StandardScaler**)와 모델 학습(**DecisionTreeClassifier**) 과정을 하나의 매끄러운 파이프라인으로 묶어서 한 번에 실행해 주는 도구.

만약 파이프라인을 사용한다면, 선행 작업(**StandardScaler**)이 끝났는지 판단하고 다음 작업(**DecisionTreeClassifier**)을 실행해주는 과정을 코드로 직접 작성해줘야 함.

```
# 1. 스케일러 따로 학습 및 변환
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 2. 모델 따로 학습 및 평가
model = DecisionTreeClassifier()
model.fit(X_train_scaled, y_train)
accuracy = model.score(X_test_scaled, y_test)
```

```
from sklearn.pipeline import Pipeline

# 컨베이어 벨트 조립: 1번 공정(scaler) -> 2번 공정(tree)
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('tree', DecisionTreeClassifier())
])

# 학습 데이터만 집어넣으면 스케일링과 모델 학습이 원터치로 진행!
pipeline.fit(X_train, y_train)
```

```
from sklearn.pipeline import Pipeline
# 1. 내가 원하는 다른 도구들을 불러온다
from sklearn.preprocessing import MinMaxScaler # 다른 종류의 스케일러
from sklearn.ensemble import RandomForestClassifier # 다른 종류의 모델

# 2. 내 입맛대로 컨베이어 벨트를 조립한다
my_pipeline = Pipeline([
    ('my_scaler', MinMaxScaler()), # 내가 고른 스케일러
    ('my_model', RandomForestClassifier()) # 내가 고른 모델
])
```

공정 단계	도구 (예시)	언제 쓸까?
1단계: 데이터 전처리 (Scaler, Imputer 등)	StandardScaler MinMaxScaler SimpleImputer	데이터의 단위를 맞출 때 데이터가 0~1 사이에 있어야 할 때 비어있는 데이터(결측치)를 채울 때
2단계: 예측 모델 (Classifier, Regressor 등)	DecisionTreeClassifier LinearRegression SVC (Support Vector Machine)	스무고개 형식으로 분류할 때 집값이나 주가 같은 연속된 숫자를 예측할 때 비선형 데이터를 분류할 때